

第 28 讲实验设计文档

PyTorch Linear、Activation 与 Transformer FFN Shape

实验目标：不是验证“PyTorch 能运行”，而是验证 `nn.Linear` 与前面手写的 affine transformation 在数值和 Shape 上完全一致，并观察 Activation、参数量与 LLM 风格三维 Tensor 的行为。

1 研究问题 (RQ)

- RQ1:** PyTorch 的 `nn.Linear` 与手写 $XW^T + b$ 是否逐元素一致？
- RQ2:** `weight.shape=(out,in)` 的存储约定是否能与课程中的矩阵公式对齐？
- RQ3:** 输入从单样本扩展到 Batch 和 (B, T, D) 后，Linear 是否仍只映射最后一维？
- RQ4:** ReLU、GELU、SiLU 是否保持输入 Shape，同时产生不同的非线性数值映射？
- RQ5:** 一个 PyTorch MLP 的参数量是否等于各 Linear Layer 参数量之和？

2 实验假设

- H1** 固定 Weight/Bias 后，`linear(X)` 与 $X @ \text{weight.T} + \text{bias}$ 最大绝对误差应接近 0。
- H2** 对 (B, T, D_{in}) 输入，`Linear(Din,Dout)` 输出应为 (B, T, D_{out}) 。
- H3** ReLU/GELU/SiLU 的输出 Shape 应等于输入 Shape，但数值不同。
- H4** 普通 ReLU/GELU 不引入新的 Weight/Bias，因此不会增加 MLP 的可训练参数数量。

3 实验变量与控制

项目	设置
主要框架	PyTorch
随机性	<code>torch.manual_seed(28)</code> ；关键数值实验使用固定 Weight/Bias
核心输入维度	$D_{in} = 3, D_{out} = 2$
已知 Weight	$\begin{bmatrix} 0.5 & -1 & 2 \\ 1 & 0.5 & -0.5 \end{bmatrix}$
已知 Bias	$[0.2, -0.3]$
比较指标	输出数值、最大绝对误差、Shape、参数量
Autograd	本讲不调用 <code>backward()</code> ；正式自动求导留到第 29 讲

4 实验 1：检查 nn.Linear(3,2) 内部参数

步骤：

1. 创建 `nn.Linear(3,2)`;
2. 打印 `weight.shape` 与 `bias.shape`;
3. 打印 `requires_grad`;
4. 用 `named_parameters()` 检查参数名与元素数量;
5. 与理论参数量 $3 \times 2 + 2 = 8$ 对照。

预期：

$$\text{weight.shape} = (2,3), \quad \text{bias.shape} = (2,)$$

总参数量为 8。

5 实验 2：手写公式与 nn.Linear 数值对齐

步骤：

1. 用 `torch.no_grad()` 把参数替换成固定值;
2. 输入 $x = [1, 2, -1]$;
3. 手工计算 $x @ \text{weight.T} + \text{bias}$;
4. 再调用 `linear(x)`;
5. 计算最大绝对误差。

理论结果：

$$y = [-3.3, 2.2]$$

如果两种实现最大绝对误差为 0 或极接近 0，则 H1 得到支持。

6 实验 3：二维 Batch Shape

构造：

$$X.\text{shape} = (3,3)$$

经过 `Linear(3,2)`:

$$Y.\text{shape} = (3,2)$$

并再次比较模块实现与手工矩阵乘法。

7 实验 4: LLM 风格三维 Tensor

构造:

$$X.shape = (2, 4, 3)$$

表示: 2 条序列、每条 4 个 Token、每个 Token 3 个特征。

经过:

$$\text{Linear}(3, 2)$$

预期:

$$Y.shape = (2, 4, 2)$$

如果实验成立, 就直接说明 Transformer hidden states 可以不先手工展平 (B, T) 两个维度, 而让 `nn.Linear` 自动在最后一维上执行同一套映射。

8 实验 5: Activation 数值与 Shape

输入:

$$z = [-3, -1, 0, 1, 3]$$

分别计算:

- ReLU;
- GELU;
- SiLU。

记录每种输出数值和 Shape, 并比较 `nn.ReLU()` 与 `F.relu()`。

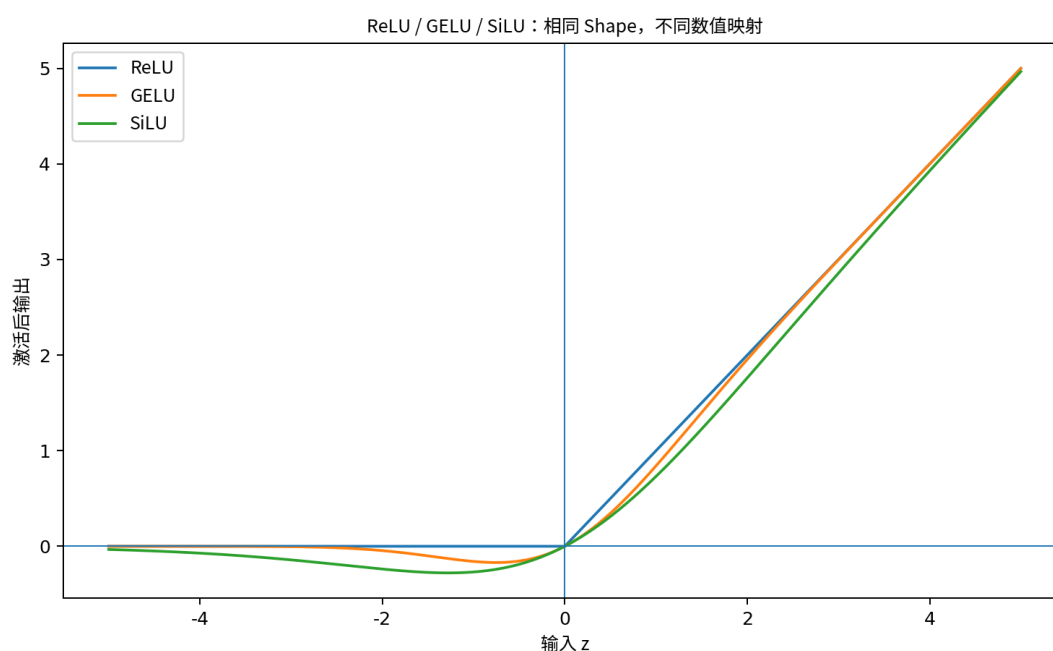


图 1: 预期观察: 三种激活对相同输入产生不同非线性映射, 但 Shape 保持一致。

9 实验 6: Linear + ReLU 最小非线性块

建立:

```
block = nn.Sequential(  
    nn.Linear(3, 2),  
    nn.ReLU(),  
)
```

固定 Linear 参数后, 记录:

1. Linear 输出;
2. ReLU 输出;
3. 整个 block 参数量。

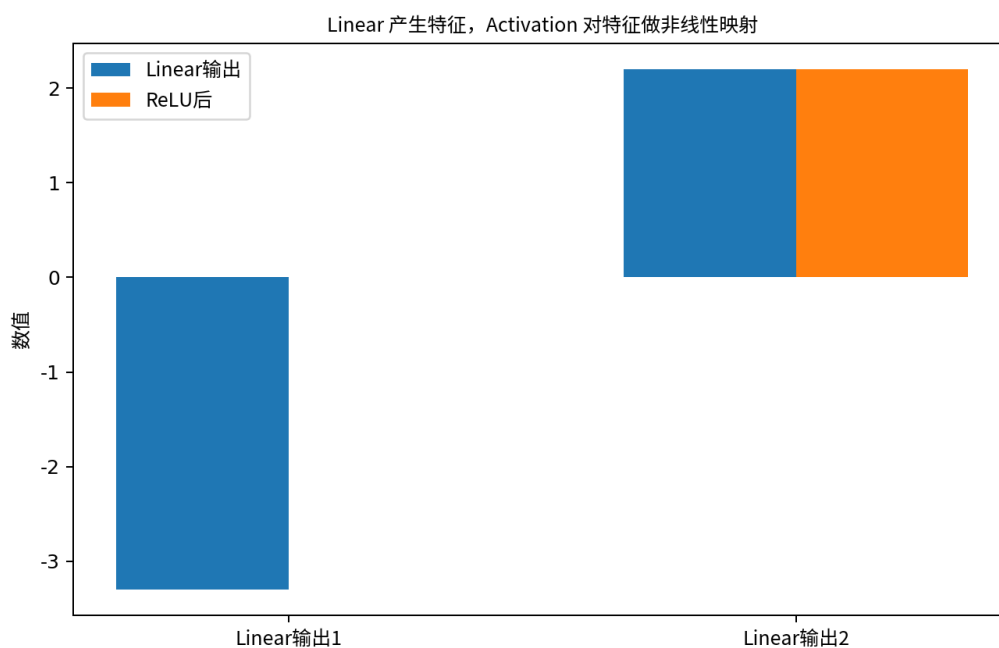


图 2: 固定输入下, Linear 先产生正负特征, ReLU 再进行非线性截断。

10 实验 7: nn.Sequential、state_dict 与参数量

建立:

$3 \rightarrow 4 \rightarrow 2$

MLP:

```
mlp = nn.Sequential(  
    nn.Linear(3, 4),  
    nn.GELU(),  
    nn.Linear(4, 2),  
)
```

理论参数量：

$$(3 \times 4 + 4) + (4 \times 2 + 2) = 26$$

检查 `state_dict()` 中应只有两个 Linear 的 Weight/Bias。

11 实验 8：Transformer FFN 风格 Shape

使用小型结构：

$$D = 4, \quad H = 8$$

输入：

$$(B, T, D) = (2, 3, 4)$$

执行：

$$\text{Linear}(4, 8) \rightarrow \text{GELU} \rightarrow \text{Linear}(8, 4)$$

预期 Shape：

$$(2, 3, 4) \rightarrow (2, 3, 8) \rightarrow (2, 3, 8) \rightarrow (2, 3, 4)$$

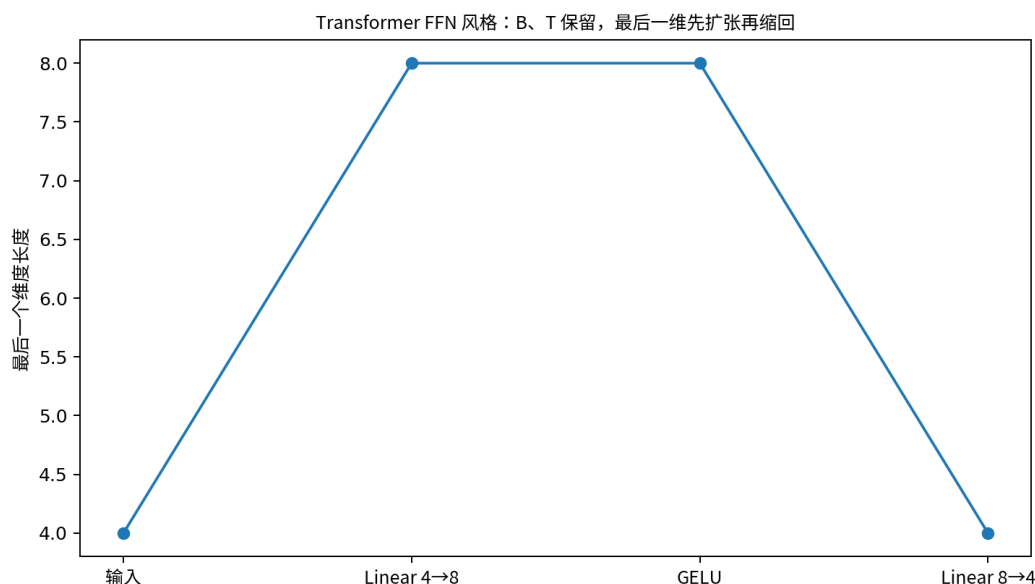


图 3: Transformer FFN 风格 Shape：B、T 保留，最后一维先扩张再缩回。

12 成功判据

实验至少满足：

1. 手工公式与 `nn.Linear` 最大误差为 0 或浮点误差级；
2. 二维与三维输入输出 Shape 符合推导；
3. Activation Shape 不变；
4. 参数量与理论计算一致；
5. `state_dict` 中参数 Shape 与结构一致。

13 科研式记录表

实验	预期	实际观察
E1 参数 Shape	$(2, 3)$ 与 $(2,)$	运行脚本填写
E2 数值对齐	最大误差 ≈ 0	运行脚本填写
E3 Batch	$(3, 3) \rightarrow (3, 2)$	运行脚本填写
E4 LLM 3D	$(2, 4, 3) \rightarrow (2, 4, 2)$	运行脚本填写
E5 Activation	Shape 不变	运行脚本填写
E7 参数量	26	运行脚本填写
E8 FFN	$(2, 3, 4) \rightarrow (2, 3, 4)$	运行脚本填写

14 常见实验误区

1. 不固定 Weight/Bias，就拿“手算值”与随机初始化模型比较；
2. 忘记 PyTorch 的 Weight 存储 Shape 是 (out, in) ；
3. 把 ReLU 输出 Shape 不变误解成“ReLU 什么也没做”；
4. 只看输出能不能跑，不检查 Shape 与参数量；
5. 把本讲 Forward 对齐结果误解成已经验证了 Backpropagation；Autograd 是下一讲的独立实验问题。

实验结论边界

如果所有实验成功，我们只能得出：PyTorch 的 Linear/Activation 与前面学习的底层计算在当前测试中一致，Shape 和参数封装符合预期。不能由此推出某个模型架构“性能更好”，也没有比较泛化能力或优化效果。